

Compito 2: Algoritmo di Prim

Implementare e testare sui dati forniti l'algoritmo di Prim per il calcolo dell'albero di ricoprimento di costo minimo, presentato a lezione e descritto a Sezione 3.5.1 delle note del corso e a Sezione 23.2 del CLRS.

L'input del programma consiste in un grafo pesato non orientato; l'output consiste nell'albero di ricoprimento di costo minimo.

Il programma deve leggere il grafo in input da file, secondo il formato descritto sotto, e scrivere l'output su un secondo file nello stesso formato. Per lo sviluppo del programma si può utilizzare un linguaggio di programmazione a scelta.

Formato di input: Il grafo viene memorizzato in un file ASCII con il seguente formato:

- N (numero di nodi, implicitamente numerati da 0 a N-1)
- sequenza di archi con costo: ogni arco (non orientato) viene descritto dagli indici dei nodi che collega e da un numero reale che rappresenta il costo/peso dell'arco.

Ad esempio:

```
10
0 7 0.124
1 3 2.3
9 1 4.14
...
```

Note implementative:

- Si consiglia di implementare una struttura dati a liste di adiacenza per mantenere il grafo. Ricordarsi che, essendo il grafo non orientato, l'arco (i, j) va memorizzato in entrambe le liste di adiacenza dei vertici i e j . Aggiungere il costo dell'arco insieme all'informazione di adiacenza.
- Non implementare direttamente la coda con priorità: sfruttare le strutture dati messe a disposizione dalle librerie del linguaggio scelto.
- Le implementazioni standard della coda con priorità (ad esempio, in C++, oppure in Python) non supportano il cambio di priorità di elementi già in coda, che viene richiesto dall'algoritmo. Il problema può essere affrontato in due modi:
 - (1) Approccio "lazy": invece di cambiare la priorità, si rimette in coda un elemento già presente con priorità nuova; quando si estrae dalla coda un elemento con priorità obsoleta, lo si scarta (in questo algoritmo, gli elementi con priorità obsoleta sono già visitati, quindi automaticamente saltati)
 - (2) Usare una libreria aggiuntiva che implementi l'aggiornamento della chiave di priorità (ad esempio: boost per il C++; heapdict per Python).

Cosa consegnare:

- (1) Una breve relazione (1-3 pagine) che descriva il problema e l'algoritmo e discuta i dettagli implementativi.
- (2) Le soluzioni almeno per i file di input forniti.

Quando consegnare: documento e soluzioni devono essere consegnati alla prova orale d'esame. Il contenuto della relazione sarà discusso durante l'esame stesso.

Avvertenza: le implementazioni e le relazioni generate da AI sono riconoscibili e saranno giudicate negativamente.